IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Low-Level Design

for

Design and Development of Identity Security Framework for EdTech

Jesteena Mary Oommen

23BCCED016

BCA in Cyber Security, Ethical Hacking & Digital Foreignsic with IBM

Yenepoya Institute of Arts, Science, Commerce and Management

Mangalore

11-04-2026

Under the guidance of

Mr. Vikyath

Lecturer, Computer Science Department
Yenepoya Institute of Arts, Science, Commerce and Management

Mangalore

Submitted to

YENEPOYA
(DEEMED TO BE UNIVERSITY)
Recognized under Sec 3(A) of the UGC Act 1956
Accredited by NAAC with 'A' Grade

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

BALMATTA, MANGALORE

YENEPOYA (DEEMED TO BE UNIVERSITY)

Table of Content

1. Introduction

1.1 Scope of the Document

This document provides the complete technical specifications for the Identity Security Framework for EdTech. It details:

- The internal logic of all authentication modules (Bcrypt, Google OAuth, WebAuthn, TOTP) tailored for academic environments.
- The **identity restoration protocol** – a Combined Dual-Factor Handshake requiring both cipher fragments (first/last 4 characters of the original password) and a valid TOTP code.
- The SQLite database schema, including field-level encryption, indexing, and relationship mappings for student, teacher, and admin entities.
- API contracts for every security-sensitive endpoint, with request/response formats and HTTP status codes.
- Session state management, caching policies, and non-functional requirements (security, performance, compliance with FERPA and student data protection).

The document covers the **prototype implementation** (Flask + SQLite) intended for demonstration, testing, and compliance validation within educational institutions.

1.2 Intended Audience

- **Developers and Security Architects –** to implement and review the authentication logic, fragment extraction, and quarantine mechanisms for campus portals.
- **EdTech IT Administrators –** to understand deployment prerequisites (environment variables, seed data, backup procedures) for learning management systems.
- **Educational Compliance Auditors –** to map controls to FERPA, COPPA, and other student data privacy regulations.

1.3 System Overview

The framework is a high-security Flask application featuring:

- **Primary Authentication**: Bcrypt-hashed credentials (rounds=12) and optional Google OAuth 2.0 for institutional single sign-on.

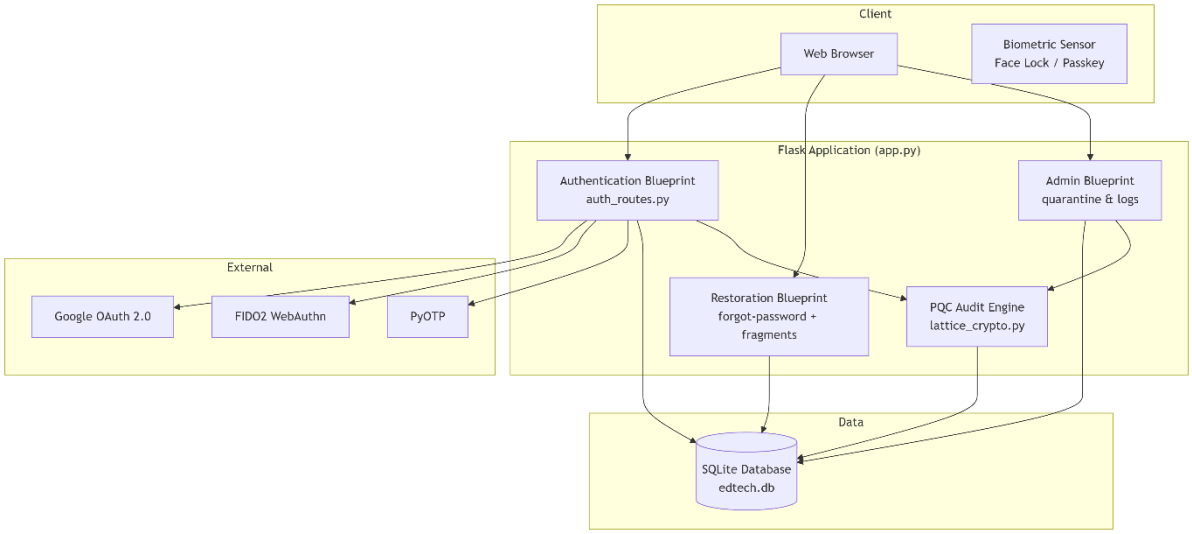
- **Secondary Authentication (MFA)**: Multi-vector WebAuthn (Passkey / Face Lock) and TOTP (6 digits, 30 sec) to protect student and faculty accounts.
- **Identity Restoration**: Combined Dual-Factor Handshake – the user must provide both **cipher fragments** (derived from the original password) and a valid TOTP code to reset a forgotten password. After 3 failures, the account is quarantined for 24 hours.

  All authentication events and restoration attempts are logged with a **post-quantum cryptography (PQC)-simulated signature** to ensure audit trail integrity for academic records.

2. System Design

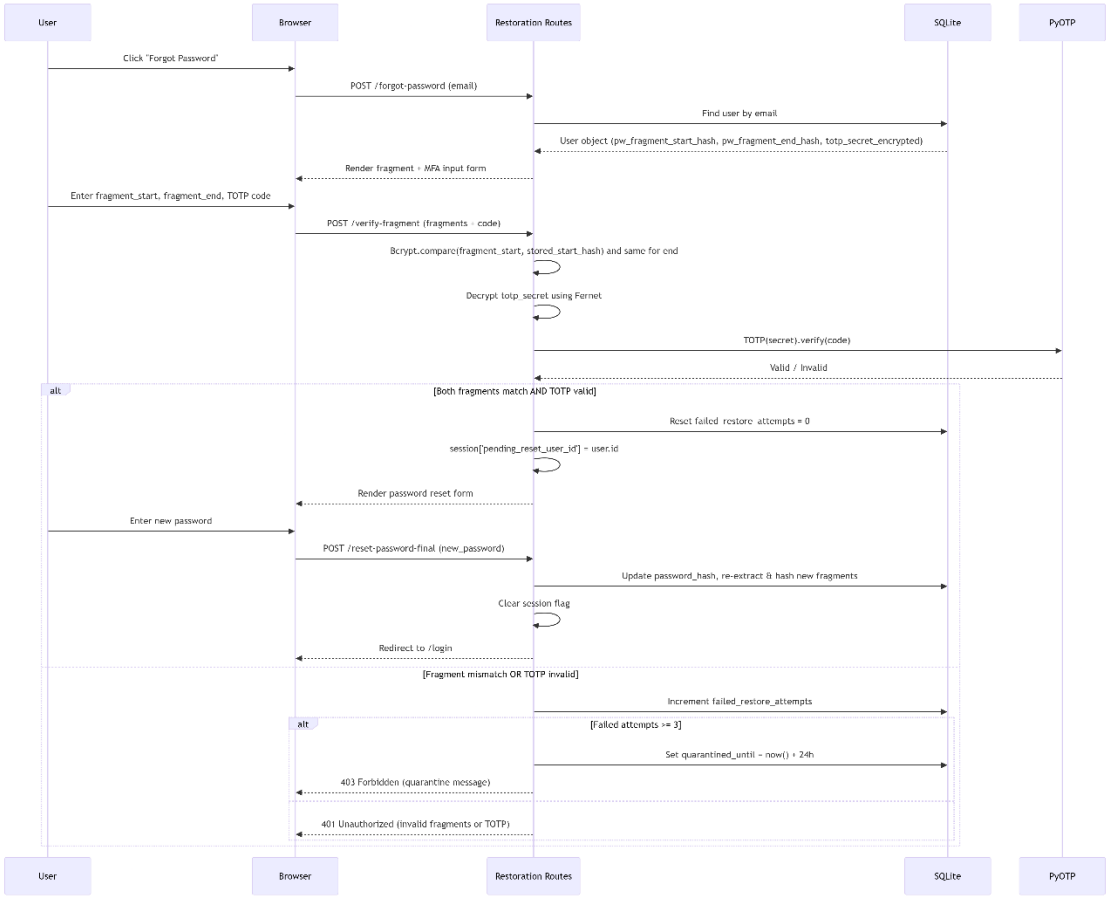
2.1 Application Design

The system uses a **Blueprint-based Flask architecture** separating Campus Access, Identity Verification, and Administrative Governance.



2.2 Process Flow

The following sequence diagram illustrates the **Dual-Factor Restoration Handshake** for students and faculty.



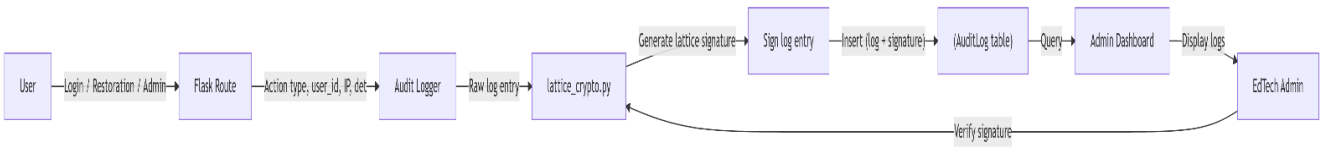
2.3 Information Flow

Every security-relevant action (login success/failure, restoration attempt, quarantine event) is logged with a PQC signature.

Component	Module	Responsibilities	Key Functions

Handshake Engine	`auth_routes.py`	Coordinates the multi-vector verification during restoration.	`forgot_password()`, `verify_fragments()`, `reset_password_final()`
Cipher Fragmenter	`crypto_utils.py`	Extracts first 4 and last 4 characters of a password; hashes with Bcrypt.	`extract_fragments(password) → (start, end)`, `hash_fragment(fragment)`
Node Quarantine	`security.py`	Tracks failed restoration attempts; enforces 24-hour lockout.	`increment_failure(user_id)`, `is_quarantined(user_id) → bool`, `reset_quarantine(user_id)`
WebAuthn Handler	`webauthn_auth.py`	Implements FIDO2 registration and authentication flows.	`begin_registration()`, `complete_registration()`, `begin_authentication()`, `complete_authentication()`
PQC Audit Engine	`lattice_crypto.py`	Generates and verifies post-quantum signatures for audit logs.	`sign_log(entry: dict) → str`, `verify_log(entry: dict, signature: str) → bool`
OAuth Handler	oauth.py	Manages Google OAuth 2.0 redirect and token exchange.	google_login(), google_callback(), refresh_token()

2.4 Components Design



2.5 Key Design Considerations

- Zero-Trust Restoration – No single factor (email, knowledge of identity, or TOTP alone) can reset a password. The combination of password fragments + TOTP is mandatory, protecting student and faculty accounts from social engineering.

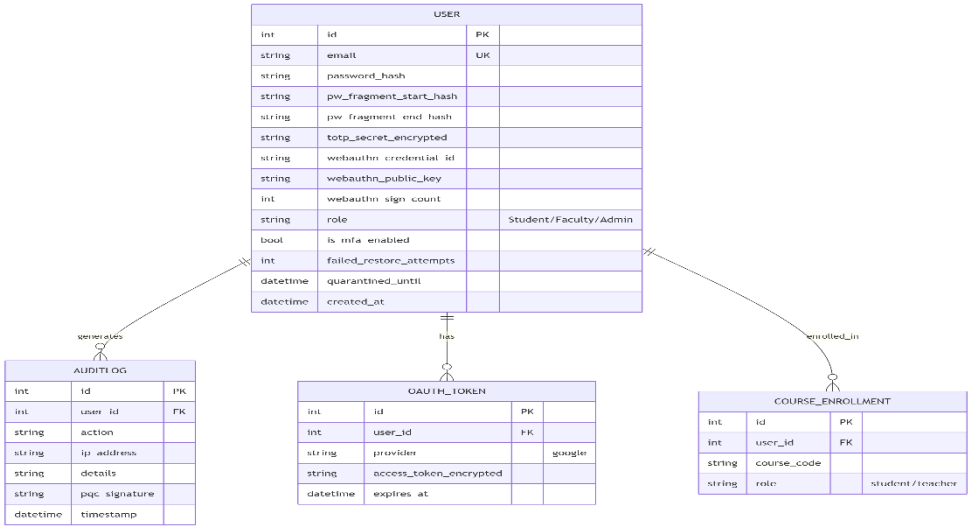
- Quarantine to Prevent Brute Force – After 3 failed restoration attempts, the account is locked for 24 hours. This makes online brute-forcing of fragments (only 4 characters each) or TOTP (6 digits) infeasible.
- Defense in Depth for Fragments – Even if the database is compromised, only Bcrypt hashes of the first and last 4 characters are exposed. Full password remains secret, preserving academic record integrity.
- Post-Quantum Audit Trail – Simulated lattice signatures ensure any retrospective alteration of audit logs (e.g., covering up a grade change) is detectable.
- Session Isolation – The pending_reset_user_id flag is stored only in the server-side session, not in a cookie, and expires after 15 minutes of inactivity.

2.6 API Catalogue

Endpoint	Method	Authentication	Request Body
/login	POST	None	{email, password}
/login/webauthn/begin	POST	None	{email}
/login/webauthn/complete	POST	None	{credential}
/forgot-password	POST	None	{email}
/verify-fragment	POST	Rate-limit (5/min)	{fragment_start, fragment_end, totp_code}
/reset-password-final	POST	Session pending_reset_user_id	{new_password}
/admin/quarantine/list	GET	Admin role	–
/admin/quarantine/clear/<id>	POST	Admin role	–

3. Data Design

3.1 Entity-Relationship (ER) Diagram



3.2 Data Model (Schema & Encryption)

Field	Protection	Purpose	Field
password_hash	Bcrypt (12 rounds)	Primary credential storage	password_hash
pw_fragment_start_hash / pw_fragment_end_hash	Bcrypt (10 rounds)	Password fingerprints for restoration	pw_fragment_start_hash / pw_fragment_end_hash
totp_secret_encrypted	AES-128 (Fernet)	TOTP shared secret	totp_secret_encrypted
access_token_encrypted	AES-128 (Fernet)	OAuth token storage	access_token_encrypted
pqc_signature	Lattice-based (simulated)	Audit log tamper detection	pqc_signature
All other fields	Plain text	No sensitive data (email, role, timestamps, counters)	All other fields

3.3 Data Access Mechanism

SQLAlchemy ORM provides abstracted access to edtech.db. Raw SQL queries are prohibited to prevent injection attacks. All database interactions are parameterized via SQLAlchemy Core or ORM methods.

3.4 Data Retention & Archival Policies

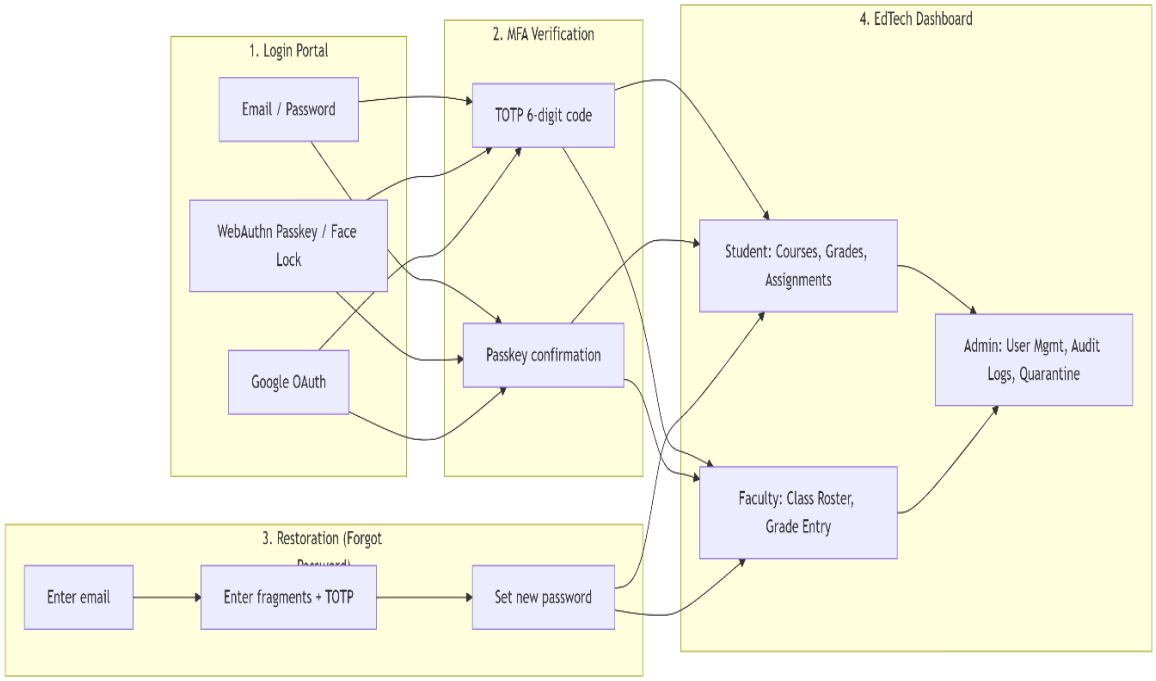
Entity	Retention	Enforcement	Entity	Retention
`AuditLog`	7 years (FERPA recommendation)	Monthly export to encrypted cold storage, then deletion	`AuditLog`	7 years (FERPA recommendation)
`User`	Until account deletion + 30 days	Soft delete with `is_active=False`, then hard delete	`User`	Until account deletion + 30 days
`OAuthToken`	Until expiry + 90 days	Cleanup job removes expired tokens	`OAuthToken`	Until expiry + 90 days
`CourseEnrollment`	Indefinite (academic record)	Never deleted; only archived	`CourseEnrollment`	Indefinite (academic record)
Failed restoration attempts	Reset after successful login or quarantine expiry	Automatic	Failed restoration attempts	Reset after successful login or quarantine expiry

3.5 Data Migration & Seeding

- **Initial seed:** seed_edtech.py creates an admin user (admin@edtech.edu), test faculty (professor@edtech.edu), and test students (student1@edtech.edu, student2@edtech.edu).
- **Idempotency:** Script checks for existing users before insertion**.**
- **Migration:** Alembic manages schema versioning; initial migration creates all tables and indexes.

4. Interfaces

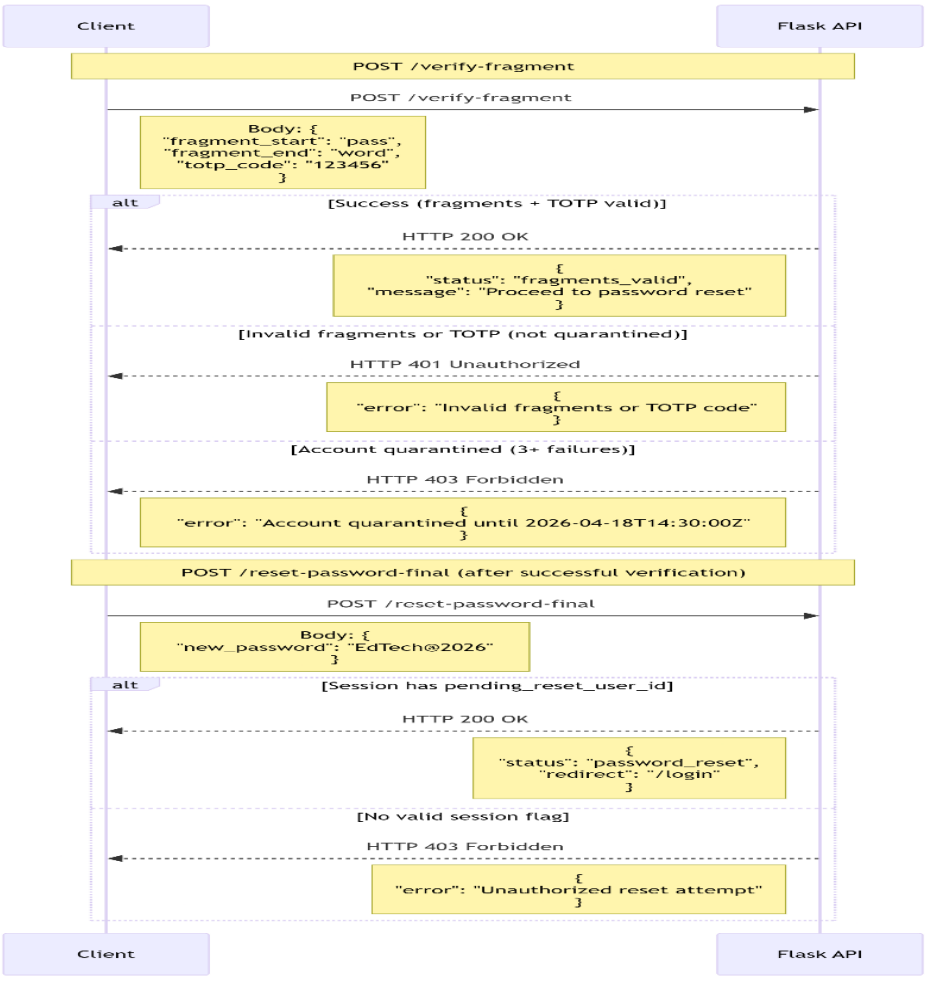
4.1 User Interface (UI) Layout



UI Specifics:

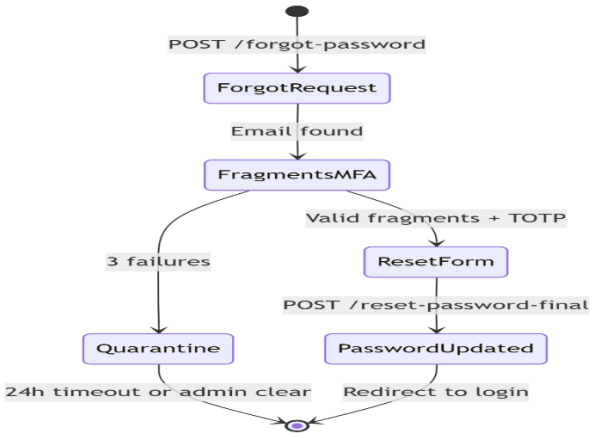
- Clean, accessible design with dark/light mode (WCAG 2.1 compliant).
- Fragment input fields are masked (show only dots).
- Quarantine warning displays remaining lockout time.
- Admin panel includes a “Clear Quarantine” button for each locked account.
- Student dashboard shows only enrolled courses; faculty dashboard shows class management.

4.2 API Contracts (Request / Response Structure)



5. State and Session Management

- **Session implementation:** Flask sessions signed with SECRET_KEY (≥32 bytes, environment variable). Session data is stored client-side in a signed cookie.
- **Session flags for restoration:**
- pending_reset_user_id – set after successful fragment+MFA verification; allows access to /reset-password-final. Expires after 15 minutes of inactivity.
- authorized_reset_user_id – temporary grant for final reset (cleared after use).
- Cookie security: Secure=True, HttpOnly=True, SameSite=Lax.
- Session timeout: 30 minutes idle; 8 hours absolute.



6. Caching

Content	Cache Policy	TTL	Reason
Static assets (CSS, JS)	Client-side, aggressive	1 year	`Cache-Control: public, max-age=31536000`
HTML templates	No caching	–	Dynamic content (quarantine status, user roles)
User session data	Server-side session	30 min	Flask session cookie
WebAuthn challenges	In-memory (server)	5 min	Prevents replay attacks
Failed attempt counters	In-memory + DB	Persistent	DB is source of truth

7. Non-Functional Requirements

7.1 Security Aspects

Control	Implementation	Verification Method
Password hashing	Bcrypt (rounds=12)	OWASP compliant
Fragment hashing	Bcrypt (rounds=10)	Separate salt per fragment
TOTP MFA	PyOTP, 6 digits, 30 sec	Manual test with Google Authenticator
WebAuthn	FIDO2 (Passkey / Face Lock)	Test with platform authenticator
Restoration lockout	3 failures → 24h quarantine	Simulated brute force
PQC audit signatures	Lattice-based (simulated)	Tamper detection test

CSRF	Flask-WTF tokens	Automated check
CSP	Flask-Talisman: `default-src 'self'`	Browser console
SQL injection	SQLAlchemy ORM	Pen test with SQLmap
XSS	Jinja2 autoescaping	Manual injection attempts
Session security	`Secure`, `HttpOnly`, `SameSite=Lax`	HTTP response headers

7.2 Performance Aspects

Metric	Target	Measurement Tool
Login (password + TOTP)	< 500 ms	Flask debug toolbar
WebAuthn authentication	< 800 ms (incl. biometric)	End-to-end timing
Fragment verification (Bcrypt)	< 100 ms	`timeit` profiling
Restoration (full flow)	< 2 seconds	User experience timing
Concurrent users	500 simulated (typical class size)	Locust load test
Audit log query (30 days)	< 200 ms	SQLite `EXPLAIN QUERY PLAN`

Database indexes (performance critical):

- User.email (unique)
- User.quarantined_until (for cleanup jobs)
- AuditLog.timestamp
- AuditLog.user_id
- CourseEnrollment.user_id

Conclusion

The Low-Level Design (LLD) v1.0 for the **Identity Security Framework for EdTech** defines a robust, tiered security architecture that ensures **secondary factor proof even during credential restoration** for students and faculty. The Combined Dual-Factor Handshake (cipher fragments + MFA) prevents unauthorized password resets, while quarantine lockout and PQC-signed audit logs satisfy FERPA and other educational data protection requirements. All critical flows – from the application architecture to API contracts – are documented with Mermaid diagrams, providing a clear implementation roadmap for developers, security architects, and educational compliance auditors.